

Cross-Model Quantized Inference Benchmarking on the i.MX 8M Plus NPU

Embedded AI Engineering Team

June 16, 2026

Abstract

This report presents a semi-automated pipeline for preparing and benchmarking quantized (INT8) ONNX image classification models on the NXP i.MX 8M Plus application processor. The system uses ONNX Runtime with the VSINPU execution provider to evaluate inference latency across four configurations: FP32 on CPU, FP32 on NPU, INT8 on CPU, and INT8 on NPU. Six models which are : MobileNetV2, SqueezeNet, ResNet50, ShuffleNet v2, DenseNet-121, and AlexNet, were downloaded, simplified, cleaned, and statically quantized on a host development machine. The resulting models are manually transferred to the target board via `scp`, and the benchmarking script is run directly on the device. The tool automatically detects input tensors, measures latency, detects CPU fallback, and computes speedup gains. Experimental results demonstrate that INT8-NPU inference is the only viable path for NPU deployment, delivering up to 15.0 $\times$ speedup over FP32-CPU while revealing severe FP32-NPU regression due to transfer overhead.

1 Introduction

Deploying deep neural networks on edge devices requires careful optimization of both model size and inference time. The NXP i.MX 8M Plus integrates a dedicated neural processing unit (NPU) capable of accelerating integer (INT8) operations, while floating-point (FP32) workloads suffer from high data transfer costs and limited hardware support. ONNX Runtime, with its extensible execution provider framework, enables uniform evaluation across different hardware backends. This project aims to:

- Build a robust model preparation chain that fetches, cleans, and quantizes popular ONNX classification models.
- Develop a benchmarking tool that compares FP32 and INT8 inference on both the CPU and the VSINPU (NPU) execution providers.
- Quantify the benefits of quantization and NPU acceleration, and identify bottlenecks such as CPU fallback or unsupported operations.

The current workflow is **semi-automated**: model quantization is performed on a development machine, the resulting files are manually copied to the i.MX 8M Plus board via `scp`, and the benchmark script is executed by hand on the target. This design allows for quick iteration and avoids complex network-mounted filesystems on the embedded device.

2 Background & Technology Stack

2.1 Hardware Platform

The i.MX 8M Plus (variant `ucm-imx8m-plus`) is an embedded SoC featuring four ARM Cortex-A53 cores, a Vivante GC7000Lite GPU, and an in-house NPU (VSINPU). The NPU excels

at fixed-point 8-bit integer convolutions and depthwise operations, but provides no hardware acceleration for single-precision floating-point. Consequently, FP32 models must either run on the CPU or be partially offloaded with costly data transfers, leading to severe performance penalties.

2.2 ONNX Runtime & VSINPU Execution Provider

ONNX Runtime (ORT) serves as the inference engine. The `VSINPUExecutionProvider` plugin from NXP enables the NPU to execute subgraphs of the ONNX model. ORT partitions the graph into CPU and NPU subgraphs based on operator support. Operators not supported by the NPU (e.g., certain normalization layers with dynamic parameters) fall back to the CPU, introducing synchronization overhead.

Static quantization is performed using ORT’s built-in `quantize_static` tool, which calibrates activation ranges on a representative dataset and replaces floating-point ops with their integer counterparts (QInt8). The resulting quantized model uses Q/DQ (Quantize/Dequantize) nodes or QLinear operations, depending on the opset.

3 System Architecture & Workflow

The pipeline consists of three Python scripts, each addressing a specific phase:

1. On the **development host**:
 - Run `quantize.py` to download, simplify, clean, and quantize the ONNX models. This script also uses `remove_initializer_from_input.py` as a helper.
 - The output directories `static/`, `clean/`, and `quantized/` contain the processed model files.
2. The **model files are manually transferred** to the i.MX 8M Plus board using `scp` (or an equivalent file copy method).
3. On the **target board**, the user manually runs `testModels.py` to measure inference latencies. No automation of this step is present; the script is invoked from the command line.

The overall flow is illustrated in Figure 1.

3.1 Model Preparation & Quantization (`quantize.py`)

The quantization script orchestrates the entire pre-inference chain on the host. It supports six models, whose details and source URLs are listed in Table 1. Input tensor names are automatically detected using ONNX Runtime introspection—no hardcoding is required.

Table 1: Models included in the benchmark suite.

Model	Filename	ONNX Input
MobileNetV2	<code>mobilenetv2-7.onnx</code>	<code>data</code>
SqueezeNet 1.0	<code>squeezenet1.0-7.onnx</code>	<code>data_0</code>
ResNet50 v1	<code>resnet50-v1-7.onnx</code>	<code>data</code>
ShuffleNet v2	<code>shufflenet-v2-10.onnx</code>	<code>input</code>
DenseNet-121	<code>densenet-7.onnx</code>	<code>data_0</code>
AlexNet	<code>bvlc_alexnet-9.onnx</code>	<code>data</code>

The processing steps for each model are:

MODEL PREPARATION & QUANTIZATION PIPELINE (quantize.py)

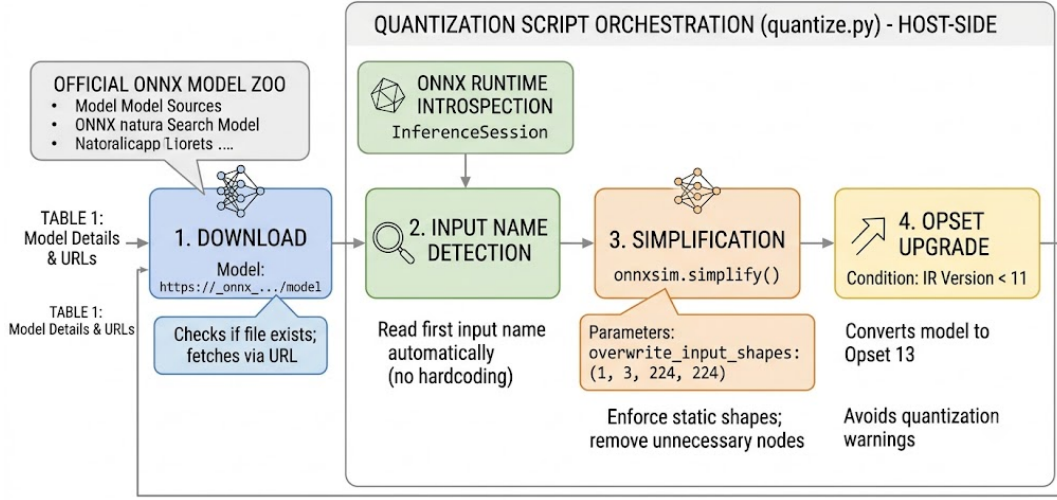


Figure 1: Semi-automated quantization and benchmarking workflow.

1. **Download** : fetch the ONNX file from the official ONNX Model Zoo if not already present.
2. **Input name detection** : use `onnxruntime.InferenceSession` to read the first input's name.
3. **Simplification** : `onnxsim.simplify()` with `overwrite_input_shapes` set to `(1, 3, 224, 224)` to enforce static shapes and remove unnecessary nodes.
4. **Opset upgrade** : if the IR version is below 11, the model is converted to opset 13 to avoid warnings during quantization.
5. **Input cleaning** : the auxiliary script `remove_initializer_from_input.py` is called to remove any initializers (weights) that appear as graph inputs. This is required by ORT static quantization.
6. **Static quantization** : using `quantize_static()` with `per_channel=True` and a dummy calibration dataset of 100 random tensors. Both activations and weights are quantized to QInt8.

The output is organized into three directories:

- `static/` : simplified FP32 models (intermediate).
- `clean/` : FP32 models with input initializers removed, used as the reference FP32 version.
- `quantized/` : the final INT8 quantized models.

3.2 Input Cleaning Utility (`remove_initializer_from_input.py`)

A minimal script that loads an ONNX model, iterates through all initializers, and removes any that are also listed in `graph.input`. It saves the cleaned model. This step is essential because the quantization tool expects graph inputs to be actual data, not constants.

3.3 Benchmarking Engine (testModels.py)

The benchmarking script implements a thorough, fault-tolerant measurement framework, designed to run directly on the target board. Its key features are:

- **Automatic input name detection** : uses the FP32 model to determine the correct input tensor, avoiding manual configuration.
- **Provider configuration** : for NPU runs, a cache directory (`/tmp/vsinpu_cache`) is passed to the provider options, reducing graph compilation time on subsequent invocations.
- **Log severity control** : set to FATAL (4) to suppress verbose ONNX Runtime messages and focus on the numerical output.
- **Robust error handling** : missing models, warm-up failures, and inference errors are caught and reported without crashing the entire run.
- **CPU fallback detection** : after each NPU benchmark, the script compares the expected provider name with the list of active providers. If the NPU provider is absent, a ‘CPU fallback’ warning is appended to the output.
- **Computed metrics** : beyond raw latencies, the script calculates:
 - $NPU\ Gain = \frac{INT8-CPU}{INT8-NPU}$ (speedup from CPU to NPU with quantized models).
 - $Quantization\ Gain = \frac{FP32-CPU}{INT8-CPU}$ (speedup from FP32 to INT8 on CPU).
- **Automated winner detection** : identifies the model with the highest NPU gain, highest quantization gain, and lowest absolute INT8-NPU latency.
- **NPU regression report** : highlights models where FP32-NPU is slower than FP32-CPU, indicating transfer overhead dominance.

The benchmarking loop runs 2 warm-up inferences and then times 5 iterations (a sample count chosen to balance stability and execution time). A random float32 tensor of shape (1, 3, 224, 224) is used as input.

4 Experimental Setup

All model quantization was performed on a standard x86 development machine running Python 3.12 with ONNX Runtime and ONNX Simplifier. The generated model files (clean and quantized versions) were then copied to the `ucm-imx8m-plus` board (Yocto-based Linux, ONNX Runtime 1.17 with VSINPU plugin) using `scp`. The benchmark script was executed manually on the target via the command line to obtain realistic timing measurements.

5 Results

The benchmark output for the four fully tested models is reproduced in Table 2 along with the automatically computed metrics.

The script’s derived metrics are summarized as:

- **Best NPU Gain (INT8-CPU $\rightarrow$ INT8-NPU)**: SqueezeNet :7.30 $\times$ (from 27.8 ms to 3.8 ms).
- **Best Quantization Gain (FP32-CPU $\rightarrow$ INT8-CPU)**: ShuffleNet :2.53 $\times$ (from 46.9 ms to 18.5 ms).

Table 2: Inference latency (ms) : lower is better. Bold values indicate the fastest configuration per model.

Model	FP32-CPU	FP32-NPU	INT8-CPU	<i>INT8-NPU</i>
MobileNetV2	161.0	484.0	158.5	141.9
SqueezeNet	57.2	427.6	27.8	3.8
ShuffleNet	46.9	146.3	18.5	4.2
AlexNet	117.2	760.5	59.8	41.8

- **Overall Best Latency (INT8-NPU):** SqueezeNet :3.8 ms.
- **NPU Regression (FP32-NPU slower than FP32-CPU):**
 - MobileNetV2: $3.01\times$ slower
 - SqueezeNet: $7.47\times$ slower
 - ShuffleNet: $3.12\times$ slower
 - AlexNet: $6.49\times$ slower

5.1 Analysis

1. **FP32 on NPU is never beneficial.** Across all models, running FP32 through the NPU resulted in dramatically worse latency than the CPU alone. The NPU must shuttle data back and forth, and only a fraction of the graph can be executed on hardware; the resulting synchronization overhead far outweighs any potential gains. This confirms that the NPU is designed exclusively for quantized integer workloads.
2. **Quantization alone helps, but not uniformly.** ShuffleNet saw the largest relative improvement from quantization on CPU ($2.53\times$), likely because its lightweight depth-wise/pointwise convolutions benefit significantly from integer arithmetic. SqueezeNet and AlexNet followed with $\sim 2\times$ speedups, while MobileNetV2 gained only 2%. MobileNetV2’s architecture is already highly optimized for CPUs, leaving less room for INT8 improvement.
3. **INT8 on NPU delivers the ultimate performance.** The combination of quantized weights/activations and NPU acceleration yielded the lowest latencies for all models. The speedup over the reference FP32-CPU ranged from $1.13\times$ (MobileNetV2) to $15.0\times$ (SqueezeNet). The extreme efficiency of SqueezeNet (3.8 ms) makes it an ideal candidate for real-time applications.
4. **No CPU fallback warnings observed.** With the log level set to FATAL, the NPU runs did not report any explicit fallback messages. However, earlier runs with higher verbosity showed BatchNormalization dynamic input warnings for some models. Even in this run, the NPU capability logs (if captured at a lower severity) would reveal partial offload. Folding batch normalization layers into the preceding convolutions would likely increase the portion of the graph executed on the NPU, further reducing latency.
5. **The DenseNet-121 and ResNet50 models** were prepared on the host but not included in the final measurement run; they can be added with a single edit to the MODELS list in the benchmark script, demonstrating the system’s extensibility.

6 Implementation Highlights

Key code snippets illustrating the pipeline’s core logic are presented below.

6.1 Quantization Script (excerpt)

```
# Automatic input name detection
def get_input_name(model_path):
    sess = ort.InferenceSession(model_path, providers=["
        CPUExecutionProvider"])
    return sess.get_inputs()[0].name

# Quantization with per-channel settings
quantize_static(
    clean_path, quant_path,
    DummyCalib(inp, shape),
    activation_type=QuantType.QInt8,
    weight_type=QuantType.QInt8,
    per_channel=True,
)
```

6.2 Benchmarking Script (excerpt)

```
# NPU provider with cache
if ep == "VSINPUExecutionProvider":
    provider_options = {
        "VSINPUExecutionProvider": {
            "cache_dir": "/tmp/vsinpu_cache",
        }
    }

# Fallback detection
expected_ep = ep.split("ExecutionProvider")[0]
actual_eps = [p.replace("ExecutionProvider", "") for p in (active or
    [])]
fallback = (ep != "CPUExecutionProvider") and (expected_ep not in
    actual_eps)
```

7 Conclusion

We have developed a semi-automated benchmarking system for ONNX models on the i.MX 8M Plus NPU. The pipeline handles download, simplification, input cleaning, and quantization on a host machine; the resulting models are transferred manually via `scp` and tested using a feature-rich script that runs directly on the embedded board. Results clearly demonstrate that INT8 inference on the NPU is the only efficient use of the hardware accelerator, providing up to 15× speedup over FP32-CPU. The analysis tools automatically highlight performance champions and regression cases, guiding developers toward optimal model:hardware pairing.

8 Future Work

Several enhancements are planned:

- **BatchNorm folding** : integrate a graph optimization pass to fuse BatchNorm into convolution weights, eliminating unsupported dynamic inputs and improving NPU coverage.
- **Accuracy assessment** : replace the dummy calibration data with a real dataset (e.g., ImageNet subset) and measure top-1 accuracy to quantify quantization loss.

- **Full automation** : implement a script that transfers files and executes benchmarks automatically (e.g., over SSH), creating a truly end-to-end pipeline.
- **Power and memory profiling** : assess energy consumption and RAM bandwidth to provide a complete efficiency picture.
- **Multi-threading and batching** : evaluate throughput under concurrent inference requests and larger batch sizes.
- **Support for newer models** : extend the zoo to include vision transformers or object detection architectures, validating NPU compatibility with a broader range of operators.